

Electrical Problem Statements

Seven problems, one theme: the robot only works if the electrical layer underneath it is correct, measured, and designed to survive its own faults.



SCAN TO JOIN THE
WHATSAPP GROUP

- 01** **Motors, Drivers and Electrical Noise** **MEDIUM** **PAGE 4**

Actuation & power — what each motor actually takes as an input, whether it can move your robot, and a bench experiment on why electrical design creates software bugs.
- 02** **Encoders, Odometry and Sensor Characterization** **HARD** **PAGE 7**

Sensing & odometry — counts to v and ω , and the intrinsic calibration everyone skips before blaming the software team.
- 03** **Control: PID, MPC and Knowing Which to Reach For** **HARD** **PAGE 9**

Control systems — tune it, break it with saturation, discretize it, then do the same job with MPC and compare. Simulation, with plots.
- 04** **Signals and Buses: PWM, UART, I2C, SPI, CAN** **MEDIUM** **PAGE 11**

Interfacing — which bus belongs where, what sits on each end, and how to wire it across STM32, Arduino and ESP32 without it failing on the robot.
- 05** **Power Budget, Distribution and Protection** **HARD** **PAGE 13**

Power architecture — a complete robot to feed, every number calculated, and protection layered so one fault costs one component instead of all of them.
- 06** **Filtering at the Edge, and the Delay It Costs You** **HARD** **PAGE 16**

Signal processing — clean up a noisy sensor on the microcontroller, then measure what the resulting lag does to the controller upstream.
- 07** **Nested Control on a Quadcopter** **VERY HARD** **PAGE 19**

Flight control — four loops stacked inside each other, tuned innermost first, then broken on purpose to show why the order matters. Rotor failure as an extension.

How to Approach These Problems

These are not exam questions with a marking key. Every one of them can be answered shallowly, and every one of them rewards going further than the question strictly asked for. We can tell the difference in about two minutes of conversation.

WHAT WE ARE ACTUALLY TESTING

Whether you can be trusted with the electrical layer of a competition robot. That means numbers with units behind them, decisions you can defend, and an honest account of what you measured versus what you assumed. A confident answer built on a guessed number is worse than no answer.

Several of these ask you to calculate something, and several ask you to measure something. Do both properly. A calculation with the working shown and the assumptions written down is worth far more than a final figure on its own, because we can see where your reasoning goes when the assumptions change.

IF YOU DO NOT HAVE THE HARDWARE

Some of these are better with a bench, a scope and a motor in front of you. If you cannot get that, say so clearly and do the work anyway: design the experiment in full, predict what each step would show, and be specific about the prediction. A well-designed experiment you could not run beats a vague account of one you did.

USING LLMs

LLMs and AI tools are allowed on every problem here. Use them, to learn a topic quickly or to get unstuck. One condition: you own everything you submit. If a number is in your report, you know where it came from and what it means.

Electrical work punishes this more than software does. A model will happily hand you a wire gauge, a fuse rating or a set of PID gains that looks entirely reasonable and is wrong for your case, and nothing will tell you until something is on fire. Check every number against a datasheet or a calculation you did yourself.

HOW YOU SUBMIT

One GitHub repository link. No slides. Reports, calculations, diagrams, scope captures, MATLAB files, everything goes in there. Hand-drawn diagrams photographed and committed are completely acceptable. Where a problem asks for a demo or a bench result, add a short edited video showing it working.

Motors, Drivers and Electrical Noise

Start from the fundamentals, but keep it problem-oriented. The aim is not to recite motor types. It is to be the person who can say which motor this robot needs, whether it will actually move it, and why the encoder wire started lying the day the steppers went in.

TYPE	COVERS	PLUS	OUTPUT
Study and short report	DC, BLDC, stepper, servo	Drivers, gearboxes, EMI, grounding	Notes, one sizing calculation, three selections

1 Why This Comes First

Most arguments between the electrical and software teams start at the motor. Software sends a number, the wheel does not turn at the speed anybody expected, and both sides are certain the other is wrong. Usually nobody wrote down what that number was meant to mean.

A stepper does not take an RPM. A hobby servo does not take a speed at all. A brushed DC motor takes a voltage, and what you get out depends on the load it is fighting. Until that is second nature you cannot design a drivetrain, and you cannot tell software what to send you.

2 What to Study

This is the list, not a syllabus to tick off. Some of it you will read in an afternoon, some will take real digging. Go deeper wherever you find you cannot explain something out loud.

THE MOTORS THEMSELVES

- Brushed DC, BLDC, stepper, servo. How each one produces torque, and how each is commutated.
- **What each actually accepts as an input** — PWM, direction pin, step and direction pulses, a pulse-width angle command, CAN, UART. Be exact about which motor takes which.
- Rated voltage, current, rated torque, stall torque, no-load RPM, K_v .
- **Continuous versus peak current**, and what a duty cycle rating means in practice.
- The torque-speed curve. No-load speed, stall torque, maximum power, and where a sane design sits on it.
- **Open-loop control**. A stepper commanded to move 200 steps that you simply trust has moved 200 steps. What you gain (no sensor, no tuning) and exactly what you are betting on.
- **Closed-loop control**. The same motor with an encoder on it, correcting to a measured position. What it costs in parts and complexity, and what class of failure it removes entirely.
- Which motors give you the choice, which force one on you, and what a “closed-loop stepper” actually is.

DRIVING AND GEARING THEM

- Motor driver selection. H-bridge, ESC, stepper driver. Current rating, thermal limits, and what headroom you need over the motor's stall current.
- **Microstepping**. How intermediate current ratios between the two phases place the rotor between full steps. What it buys you in smoothness and resolution, what it costs you in holding torque per microstep, and why 1/256 microstepping does not give you 256 times the useful positional accuracy.
- Gearbox selection. What a reduction does to torque, speed, backlash, back-drivability and your effective encoder resolution.
- **How to calculate whether a motor can actually move a given robot**. Mass, wheel radius, gradient, rolling resistance, acceleration, and the current that comes out the other end.

WHEN IT GOES WRONG

- Why a stepper loses steps, and why you cannot tell that it has.
- What happens when a motor is overloaded, thermally and electrically.
- Back-EMF, flyback, and why switching an inductive load creates voltage spikes that reach the rest of your board.

- Grounding, common ground, star grounding, ground loops, and power distribution across a robot.
- EMI and EMC. How switching currents couple into nearby conductors, magnetically and capacitively.
- **Why encoder and communication wires must not casually run alongside high-current motor wires.**
- Shielding (and which end you ground), twisted pairs, ferrites, filtering, decoupling and bulk capacitors, opto-isolation.
- What happens when a stepper or a motor driver switches hard next to a sensitive signal line. Phantom encoder counts, a locked I2C bus, a microcontroller that resets for no visible reason.

THE UNIT QUESTION

For every motor type, finish this sentence precisely: “To make this move, the microcontroller sends _____, in units of _____, through _____, and the resulting motion is _____.” If you cannot fill all four blanks without hedging, you do not know that motor yet.

3 The Noise Experiment

This is the part of the problem statement we care most about, because it teaches the thing that is almost impossible to learn from reading: **electrical design creates what everybody calls software bugs.**

Build a setup where a signal line runs close to a high-current motor cable. An encoder output, an IMU on I2C, or a UART link — any of the three works. Run the motor, ideally a stepper, because steppers switch hard and constantly. Then watch the signal on an oscilloscope or a logic analyser while the motor runs.

Measure a baseline first. Count the errors: miscounted encoder pulses, I2C NACKs or lock-ups, UART framing errors, whatever your setup gives you. Get a number, over a fixed run, that you can compare against.

Then work through the mitigations one at a time, re-measuring after each. Change one thing per test.

#	MITIGATION	WHAT YOU ARE TESTING
1	Cable separation	Simply moving the signal line away from the motor cable. How much distance does it take?
2	Twisted pair	Twisting the signal with its return. Why does that help?
3	Shielding	A shielded cable, grounded at one end. Then try grounding it at both and see what changes.
4	Grounding changes	Where the signal return joins the power return. Try a shared return, then a separate one joined at a single point.
5	Ferrite / filtering	A ferrite on the cable, an RC filter at the receiver. What does each attack?
6	Routing	Crossing the motor cable at right angles instead of running parallel to it.
7	Differential signalling	The same link over RS-485 or CAN instead of single-ended. Usually the largest single improvement.

Report the error count after every step, and a scope capture of the worst case and the best case. What we want out of this is not a clean result. It is you being able to say which mitigation did the most, which did nothing on your setup, and why.

IF YOU HAVE NO EQUIPMENT

Say so up front. Then design the experiment in full detail anyway: the setup, what you would measure, the order you would try things in, and what result you would predict from each step. Predict first, and be specific. A well-designed experiment you could not run beats a vague one you did.

4 The Sizing Calculation

Reading about torque is not the same as sizing a drivetrain. Do this one properly and show the working.

The objective

Reduce the errors *without touching a line of software*. No retries, no filtering in code, no error correction.

Show the arithmetic

Every number in the chain, with units. An answer with no units is not an answer.

robot mass 20 kg · four driven wheels · wheel radius 60 mm
must climb a 15° ramp · must reach 1.2 m/s on the flat

required torque per wheel? required RPM? current draw at that operating point? does your chosen motor and gearbox survive it continuously?

State your assumptions where you have to make them, rolling resistance being the obvious one. A defensible assumption written down beats a precise number you cannot justify.

5 Choosing the Motor

The real test is being asked “what should we use for this” and having an answer with reasons behind it. Work through all three. For each, pick a motor type, say what driver it needs, and state what you would measure or ask before committing.

#	REQUIREMENT
1	Drive wheels for the rover in section 4, holding a commanded speed on flat ground.
2	An arm joint that must hold position under load, without drifting, for several minutes.
3	A vertical lead-screw axis that must return to the same height repeatably, with no feedback sensor in the budget.

There is no single right answer to any of the three. There are answers you can defend and answers you cannot.

6 What to Hand In

ITEM	WHAT IT CONTAINS
Comparison table	The four motor types across input signal and units, driver required, feedback available, holding behaviour, failure mode, and where each is the obvious choice.
Sizing calculation	Section 4, worked through with units, assumptions stated.
Noise experiment	Section 3. Baseline error count, a result after each mitigation, scope captures of best and worst, and what you concluded.
Three selections	Section 5, with reasoning.
Sources	What you read, including at least two real datasheets. Say which numbers came from where.

QUESTIONS YOU SHOULD EXPECT

You said this motor draws 4 A at stall. Where did that number come from, and what does it mean for your battery and your driver? Why not a stepper for the rover wheels? Your encoder occasionally counts backwards while the robot drives straight — give us three physical causes. What does your driver do the instant you reverse direction at full speed? Where would you put the ferrite, and why there?

Encoders, Odometry and Sensor Characterization

An encoder gives you counts. A robot needs metres per second. Everything in between is the electrical team's responsibility, and when it is wrong, the software team gets blamed for it. This one goes considerably deeper than “learn encoders”.

TYPE	COVERS	CORE SKILL	OUTPUT
Study, derivation and calibration	Incremental, absolute, optical, magnetic	Counts $\rightarrow$ v and ω , with an error budget	Measured intrinsics, with numbers

1 The Argument This Prevents

Here is how it goes. The robot is told to drive one metre. It drives 1.14 m. It is told to turn ninety degrees and turns about eighty. Software says electrical is sending garbage odometry. Electrical says software is sending bad commands. Both lose a week.

Almost every time, nobody measured the effective wheel radius, or the track width, or the real counts per wheel revolution after the gearbox. Those are **sensor intrinsics**, they are electrical's job, and skipping them quietly poisons everything downstream.

THE POINT OF THIS PROBLEM

By the end you should be able to hand software a velocity and state, with numbers behind it, how accurate that number is and where the remaining error comes from. Not “it should be about right”.

2 What to Study

THE DEVICES

- Incremental versus absolute encoders. What does an absolute encoder know at power-on that an incremental one does not, and what must you do at startup without one?
- Optical versus magnetic. What kills each one in a real robot.
- Hall sensors, and where they show up other than as encoders.
- Quadrature A/B. How the phase relationship gives you direction as well as motion.
- **PPR versus CPR versus counts per revolution.** These are used sloppily and the confusion costs you a factor of four.
- X1, X2 and X4 decoding. What each counts, and what you gain and lose.

GETTING IT ONTO THE ROBOT

- Encoder mounting and mechanical coupling. Runout, misalignment, a coupling that flexes, a magnet that is off-centre or too far from the sensor.
- Gear ratio effects. Counts per revolution of the *motor* shaft is not counts per revolution of the *wheel*.
- Direction determination, and what happens the instant the direction of travel reverses.

TURNING COUNTS INTO NUMBERS YOU CAN TRUST

- Velocity estimation. Fixed time window versus time between edges, and how each behaves at very low and very high speed.
- Position estimation, and how error accumulates in it.
- Sampling frequency. What you are aliasing if you sample too slowly.
- Quantization. How much velocity resolution one count actually buys you at your sample rate.
- Noise, missing pulses, and phantom counts from a nearby driver.
- Debouncing and filtering, where it applies and where it just adds lag.
- Encoder calibration.

3 From Counts to Velocity

Derive this yourself. Do not copy it. Start from counts arriving over a time window and finish at the two numbers a differential-drive robot needs: linear velocity v and angular velocity ω .

Name every constant

Each one is a number somebody has to measure, and a place error can hide.

```
counts → motor revolutions → wheel revolutions (gear ratio, decoding factor)
→ wheel angular velocity → wheel linear velocity (effective wheel radius)
→  $v$  and  $\omega$  for the robot (track width, left and right wheels)
```

Write out every constant in that chain with its units. Then state, for your own setup, what one count is worth in millimetres, and what your velocity resolution is at your chosen sample rate.

4 Tuning the Intrinsic

This is the part people skip. Do it properly and report the numbers.

MEASURE, DO NOT ASSUME

- **Counts per wheel revolution.** Do not trust the datasheet times the gear ratio. Turn the wheel exactly ten times by hand and read the counter.
- **Effective wheel radius.** Not what a ruler says. A loaded rubber wheel deforms and rolls on a smaller radius than it measures. Drive a known straight distance several times and back the radius out of the counts.
- **Track width.** Not the centre-to-centre distance from the CAD. Spin in place through a known number of full rotations and back the effective track width out of the wheel difference.
- **Left and right separately.** A small mismatch between the two shows up as a slow curve on what should be a straight line.

Look up **UMBmark**, or an equivalent square-path calibration. It exists because those two errors, wrong radius and wrong track width, leave different and separable signatures.

Report before and after. Commanded one metre, measured X before, measured Y after. Commanded a full turn, same. Those four numbers say more about your work than any explanation will.

5 What to Hand In

ITEM	WHAT IT CONTAINS
Derivation	Counts to v and ω , worked by hand, every constant named with units, plus your millimetres-per-count and velocity resolution.
Encoder note	The types compared, and which you would put on a drive wheel versus an arm joint, with reasons.
Calibration report	Measured intrinsic, the procedure you followed, and the before and after error numbers.
Error budget	Where the remaining error comes from, roughly how big each source is, and which one you would attack next.

No hardware to hand? Say so, then do the derivation, the comparison and the error budget in full, and write the calibration as instructions precise enough that somebody else could run them without asking you a single question.

QUESTIONS YOU SHOULD EXPECT

Your encoder is 500 PPR into a 30:1 gearbox. How many counts per wheel revolution, and show the arithmetic. Why is your measured wheel radius smaller than the one from the ruler? The robot curves slightly left on a straight command — give us two electrical causes and one mechanical one. Your velocity is noisy at low speed. Why, and what did you do about it? What happens to odometry the moment a wheel slips, and can you detect it at all?

Control: PID, MPC and Knowing Which to Reach For

Everyone can quote what P, I and D stand for. Far fewer can tune a loop that has a saturating actuator, or say why their integrator wound up and drove the robot into a wall. Study the theory, then prove it in simulation.

TYPE	COVERS	DELIVERABLE	TOOLS
Study plus MATLAB / Simulink tasks	PID, anti-windup, discretization, MPC	Plots, gains, and comparison	MATLAB, Simulink, or Octave / Python

1 What to Study

FOUNDATIONS

- Open-loop versus closed-loop control, and what feedback actually buys you.
- What a plant model is, and how you would get one for a DC motor from measurements.
- Step response vocabulary: rise time, settling time, overshoot, steady-state error.
- Poles, stability, and what makes a loop oscillate.

PID, PROPERLY

- What each of the three terms does on its own, and what each one costs you.
- Tuning. Manual tuning, Ziegler-Nichols, and why the textbook methods often need adjusting on real hardware.
- **Integral windup.** Why it happens whenever the actuator saturates, what it looks like, and the standard anti-windup fixes.
- **Derivative kick** on a step change in setpoint, derivative on measurement instead of error, and filtering the derivative term.
- Actuator saturation and rate limits generally.
- **Discrete implementation.** Sample time, discretization, and what happens to a loop tuned in continuous time when you run it too slowly.
- Feedforward alongside feedback.
- Cascade control: an inner velocity loop inside an outer position loop, and why that structure is everywhere in robotics.

BEYOND PID

- State-space representation, and what it gives you that a transfer function does not.
- LQR, at least conceptually: optimal control against a cost function.
- **MPC.** Prediction horizon, control horizon, the cost function, and the thing PID fundamentally cannot do — handling constraints explicitly.
- What MPC costs you: computation every cycle, and a model good enough to predict with.
- Where each one belongs on a real robot, and where reaching for the fancy one is a mistake.

2 The Plant

Everyone works on the same plant so the results are comparable. A DC motor speed loop, simplified to first order:

$$G(s) = \omega(s) / V(s) = 1.5 / (0.35 s + 1)$$

input motor voltage, saturating at ± 12 V
output shaft speed in rad/s

for the position tasks, add an integrator: $\theta(s)/V(s) = G(s) / s$

If you would rather identify your own plant from a motor on the bench, do that instead and report how you got it. That is strictly better, and it is a harder piece of work.

3 The Tasks

Five, in order. Each one wants a plot and a sentence saying what the plot shows.

#	TASK
A	Tune a PID for speed control. Hit a step of 20 rad/s with under 10% overshoot, settling inside 0.5 s, and zero steady-state error. Report your gains, the step response, and the three metrics. Say how you arrived at the gains.
B	Break it with saturation. Command a step of 200 rad/s so the ± 12 V limit bites hard. Show the windup in the plot. Then implement anti-windup and show the same test again.
C	Discretize it. Run your controller at 1 kHz, then 100 Hz, then 20 Hz, without retuning. Plot all three. Explain what is degrading and why, then state the slowest rate you would accept.
D	Design an MPC for the same plant, with the ± 12 V input constraint declared explicitly. Show your horizons and cost weights, and what changing each one does.
E	Compare. PID against MPC on the same aggressive step from task B. Which handles the constraint better, what does the MPC cost in computation per cycle, and on a real robot which would you actually ship?

MATLAB and Simulink are the expected tools. Octave or Python with `control` and an MPC library is equally acceptable — we are marking the control work, not the licence you used.

Optional

Add a load disturbance mid-run and show how each controller rejects it. Worth doing.

4

What to Hand In

ITEM	WHAT IT CONTAINS
Code and models	Scripts or .slx files that reproduce every plot, in the repository.
Results	All five tasks, each with its plot, its numbers, and one or two sentences of reading.
Study notes	Your own notes from section 1. Short is fine, honest is required.
Judgement call	Half a page: on a real robot, which controller for which subsystem, and why.

QUESTIONS YOU SHOULD EXPECT

What does your integral term physically represent on this motor? You doubled K_d and the response got noisier — why? Show us the moment windup starts in your plot. Why did your 20 Hz version go unstable when the 1 kHz one was fine? What is your MPC actually optimising, and what happens if you halve the prediction horizon? Where on a real robot would MPC be the wrong choice?

Signals and Buses: PWM, UART, I2C, SPI, CAN

Every device on a robot talks over one of a handful of buses, each with its own wiring, its own limits and its own way of failing. Learn what they are, then be able to say which one belongs where and why the other three do not.

TYPE	BUSES	BOARDS	OUTPUT
Study and interfacing report	PWM, UART, I2C, SPI, CAN	STM32, Arduino, ESP32	Comparison, wiring, four choices

1 What to Study

PWM

- Frequency, duty cycle and resolution, and how a timer peripheral produces all three.
- Why motor PWM frequency matters: audible whine, switching losses, current ripple.
- Why a hobby servo's PWM is a completely different thing — a pulse width encoding an angle, not a duty cycle encoding power.
- Resolution versus frequency on a fixed timer clock, and the trade you are making.

UART

- Asynchronous with no shared clock. Baud rate, start and stop bits, parity, framing.
- What happens when two ends disagree on baud rate, and why a small clock error becomes a framing error.
- Point-to-point only. Flow control. TTL levels versus RS-232 versus RS-485.

I2C

- Two wires, addressing, ACK and NACK, clock stretching, repeated start.
- **Pull-up resistors.** How you size them, and what bus capacitance does to your rise time and therefore your maximum length and speed.
- Address collisions between two identical sensors, and what you do about it.
- Why an I2C bus locks up, how a device can hold SDA low forever, and how you recover.
- Why I2C across a long cable on a robot is usually a bad idea.

SPI

- Master and peripheral, MOSI, MISO, SCLK, chip select. Full duplex.
- The four modes, clock polarity and phase, and what happens when the two ends disagree.
- Why it goes much faster than I2C, and what it costs in pin count as devices multiply.

CAN

- A differential pair, and why differential signalling survives electrical noise so much better.
- Arbitration by identifier, and how that gives you priority without a bus master.
- Bit rate against bus length. **120 Ω termination at both ends**, and what a missing terminator does.
- Error frames, error counters, and what bus-off means.
- CAN-FD, and what it changes.
- Why serious robots and every car use it for distributed nodes.

MAKING THEM WORK ON REAL BOARDS

- **Logic levels.** 3.3 V on STM32 and ESP32 against 5 V on many Arduinos. What is 5 V tolerant, what is not, and how you level-shift each bus type safely.
- Common ground between any two devices that talk. What happens without it.
- Transceivers: you cannot drive CAN or RS-485 from a GPIO. What sits in between.
- Bus length, capacitance and noise. Which of these buses survives running the length of a robot chassis and which does not.
- Reading a datasheet for the timing and electrical sections, not just the pinout.

2 Choose the Bus

Four situations. For each, pick a bus, say what hardware sits between the two ends, and say what would have to change for you to pick differently.

#	SITUATION
1	An IMU sitting 3 cm from an STM32 on the same board, read at 500 Hz.
2	Four motor-controller nodes spread over a 1.5 m chassis, next to the power wiring, needing commands at 100 Hz and reporting back.
3	An ESP32 sending telemetry to a Jetson over a 30 cm cable.
4	A LiDAR streaming a few megabits per second continuously.

Then one wiring exercise. Take situation 2 and draw the full loom: conductors, twisting, shielding, where the terminators go, where the ground goes, and how it is routed relative to the motor power wires. This connects straight back to problem statement 01.

3 If You Have Hardware

Optional, and it lifts a submission a long way. Put two boards on a bus and capture it. A logic analyser or a scope on an I2C transaction, a UART frame, or a CAN message tells you more in ten minutes than a week of reading.

Then break it on purpose. Remove a pull-up and watch the edges go soft. Mismatch the baud rate by 10%. Pull one CAN terminator off. Capture each and explain what you are looking at.

4 What to Hand In

ITEM	WHAT IT CONTAINS
Comparison table	The five, across wires needed, speed, practical distance, number of devices, noise immunity, and what it needs on each end.
Four choices	Section 2, with the rejected options and your reasons.
Wiring diagram	The situation-2 loom, drawn properly, including grounding and routing.
Interfacing note	Logic levels, level shifting and transceivers across STM32, Arduino and ESP32, in your own words.
Captures	Optional. Any traces you took, with an explanation of each.

QUESTIONS YOU SHOULD EXPECT

Your I2C sensor works on the bench and fails on the robot. Give us four causes in order of likelihood. What size pull-ups did you pick and how did you arrive at that? Why does CAN survive next to a motor cable when UART does not? Two identical I2C sensors, one address — now what? What does your 5 V Arduino do to a 3.3 V STM32 pin, and what did you put in between?

Say the quiet part
For each choice, name the bus you rejected and the specific reason. That is where the marks are.

Power Budget, Distribution and Protection

Here is a complete robot. Work out what it draws, design the power architecture that feeds it, and design the protection that means one fault costs you one component instead of all of them.

TYPE	GIVEN	DELIVER	RULE
Design exercise, on paper	Full robot, 24 V battery	Budget, architecture, protection, layout	Assume the BMS will not save you

1 Why This One Exists

Last time, a power fault put 24 V somewhere it did not belong. It found its way through the ground, and everything downstream died with it, including a Raspberry Pi that was the most expensive part on the robot. One fault, and the whole electronics stack was gone.

That is not bad luck. That is an architecture with no layers in it. In a properly designed system, a fault takes out one fuse, or one isolated converter, and the expensive things upstream of it never notice.

THE RULE FOR THIS PROBLEM

Do not rely on the battery BMS. A BMS protects the cells. It does not know your 5 V rail exists, it will not react in time to save a logic device, and it is a single point of failure sitting at the wrong end of the system. Design as if it is not there.

2 The Robot

Design for exactly this. Where a number is not given, pick a real part, cite its datasheet, and use its numbers.

LOAD	NOTES
4 × drive motors	With their drivers. The dominant load, and the noisiest thing on the robot.
Jetson	The compute. The most expensive single item, and the one that must never see a fault.
LiDAR	Continuous draw, its own voltage requirement.
Camera	Usually powered off the compute, which is itself a design decision.
ESP32	Low draw, but it sits close to the motors.
Encoders × 4	Small, but their signal quality depends on how you power them.
IMU	Small, and sensitive to a noisy rail.

Source: a 24 V battery pack.

3 The Power Budget

Before any architecture, the numbers. Build a table with a row for every load and these columns:

- Operating voltage, and the tolerance it will actually accept.
- **Typical current** and **peak current**, which for a motor means stall, and for the Jetson means whatever it does under full load.
- Duty cycle. How long is it at peak, realistically?
- Power, at typical and at peak.
- Inrush at power-on, which for a capacitive load is far above its steady-state draw.

From that table, produce: total continuous power, total peak power, worst-case simultaneous draw, and the current on each rail. Then size the battery and state the expected runtime, with your assumptions written down.

The trap

Summing peak currents as if everything peaks 4 once gives you an oversized system. Assuming it never happens gives you a dead one. Say which you assumed and why.

Sizing the Conductors

Every number above has to travel down a wire or a copper trace, and both have limits. Calculate, do not guess.

- **Wire gauge per branch**, from its current, with derating for ambient temperature and for being in a bundle.
- **Voltage drop** over the actual cable length on the robot. A drop that is trivial on a bench becomes a brownout at the far end of a chassis under load.
- **PCB trace width** for a given current and acceptable temperature rise. Look up IPC-2152, and say which curve you used.
- **Connector current ratings**, per pin, including the derating for multiple adjacent pins carrying current at once.
- **Fuse ratings** that sit above the real peak but below what the wire can survive. Both conditions have to hold.
- Converter sizing: continuous rating, peak rating, efficiency, and the heat that efficiency figure implies.

Then answer the scaling question. What changes if you add four more motors? What changes if you move the pack from 24 V to 48 V at the same power? Which parts of your design scale cleanly and which have to be re-designed?

5 The Architecture

Draw the block diagram, from battery terminal to every load. It should show your rails, your converters, every protection element, and every ground.

Things it has to resolve:

- Which rails exist, and why those. What feeds the motors, and what feeds logic.
- **Where isolation goes**. This is the heart of the problem. An isolated converter means a fault on the motor side has no conductive path to the Jetson side. Decide what you isolate, and defend spending the money and the efficiency on it.
- Grounding topology. Star ground, single-point join between power and signal returns, and where exactly that point is.
- Sequencing. What powers up first, what powers down first, and what happens to a half-powered system.
- The e-stop path. What it actually cuts, and what stays alive so the robot can still tell you what happened.

6 Protection, Layer by Layer

Work through this systematically. For each protected item, name the element, its rating, what fault it catches, how fast it acts, and what it does not protect against.

LAYER	WHAT BELONGS HERE
At the battery	Main fuse or breaker, reverse-polarity protection, inrush limiting or soft-start, main disconnect.
Per branch	Individual fusing so one motor driver failing does not take the bus down. Sized against that branch, not the total.
At each converter	Input and output protection, overvoltage clamping, undervoltage lockout, and what the converter does on an output short.
Before each device	What sits immediately upstream of the LiDAR, the ESP32, and above all the Jetson. TVS diodes, eFuses, current limiting, filtering.
On the signal lines	Digital isolators or opto-isolation crossing between power and logic domains, series resistance, clamping to the rails.

THE SPECIFIC QUESTION TO ANSWER

Trace, on your own design, exactly what happens when 24 V appears on the logic ground. Where does the current go? What clamps or opens first, and how fast? Which components are still alive afterwards, and which are gone? If the answer is still “everything is gone”, the design is not finished.

Then do the same for each of these, briefly: a chafed motor wire shorting to chassis. A driver's high-side MOSFET failing shorted. A connector inserted one pin off. A ground wire coming loose while the robot is running. A converter failing with its input voltage passed straight through to its output.

7 Layout

No fabrication, no routing every trace. What we want is the **placement and the reasoning**, at the level of a board plan somebody could hand to a layout engineer.

- Board plan showing where power enters, where the high-current section sits, where the logic sits, and the boundary between them.
- Layer stack-up, and what each layer is for.
- Ground pours and return paths. For your fastest signal, trace its return path with a finger and say where it goes.
- Keep-outs under switching nodes and magnetics.
- Connector placement and orientation, including what stops somebody plugging a connector in backwards at 2 am.
- Thermal: what gets hot, where the copper is, where the airflow is.
- Serviceability. Fuses reachable, test points on every rail, an LED per rail so a dead rail is visible instead of mysterious.

Hand-drawn is completely acceptable. The reasoning is what is being marked.

8 What to Hand In

ITEM	WHAT IT CONTAINS
Power budget	The full table, totals, battery sizing, runtime, assumptions stated.
Calculations	Wire gauge, voltage drop, trace width, fuse and connector ratings, converter sizing. With units and sources.
Architecture diagram	Battery to every load. Rails, converters, isolation boundaries, protection, grounds.
Protection table	Every element: what it protects, its rating, its speed, and what it does not cover.
Fault walkthrough	Section 6. The 24 V-on-ground case in detail, then the others briefly.
Board plan	Placement, stack-up, grounding and reasoning. Sketches are fine.
Parts list	Real parts with real part numbers for the protection and conversion. Datasheet numbers, not guesses.

QUESTIONS YOU SHOULD EXPECT

Why that fuse rating and not one size up? How fast does it actually open at twice its rating, and is the Jetson still alive at that point? You isolated here and not there — defend the boundary. Where is your single ground join, and what happens if somebody adds a second one? Your LiDAR browns out only when all four motors accelerate together. Which of your numbers was wrong? Talk us through what saves the Jetson when the fault from last time happens again.

Filtering at the Edge, and the Delay It Costs You

Filtering a noisy sensor on the Arduino or the ESP is easy and it feels like a win. Then the controller upstream starts oscillating, because the beautifully smooth number it now receives is describing something that happened 40 ms ago. Every filter is a delay. Work out what that delay is worth.

TYPE	RUNS ON	FILTERS	CORE TRADE
Study, implementation, measurement	Arduino / ESP32 / STM32	MA, EMA, median, Savitzky-Golay	Noise removed against delay added

1 The Trade You Are Making

Your encoder velocity is noisy. You put a moving average over the last twenty samples on the ESP32 and the number goes beautifully smooth. Everyone is pleased. Then the drive controller starts hunting around its setpoint, and nobody can work out why, because the signal on the plot looks better than it ever has.

What happened is that a twenty-sample average reports, roughly, the average of the last twenty samples — which is centred about ten samples in the past. At 200 Hz that is 50 ms of lag. A feedback loop reacting to a 50 ms-old measurement is a feedback loop with 50 ms of extra phase lag, and phase lag is exactly what eats your stability margin.

THE QUESTION THIS PROBLEM ASKS

You are sitting at the low level, on the microcontroller, deciding how much to clean up a signal before you pass it upstream. How much noise should you remove, how much delay is that worth, and how do you decide the answer rather than guessing at it?

2 What to Study

THE FILTERS

- **Moving average** (boxcar). The simplest FIR filter. Window length, and its delay in samples.
- **Exponential moving average**, the first-order IIR low-pass. What α means, how it relates to a time constant and a cutoff frequency, and why it is so cheap to run.
- **Median filter**. Non-linear, and the right tool for isolated spikes that a linear filter smears out instead of removing.
- **Savitzky-Golay**. A least-squares polynomial fitted over a sliding window. Why it preserves the shape and height of a feature that a moving average would flatten, and the useful trick that it gives you a smoothed derivative for free.
- **FIR low-pass** generally: order, cutoff, transition width, and what higher order costs you in both computation and delay.
- **Complementary filter**, and where it earns its place on an IMU combining accelerometer and gyroscope.
- **Kalman filter**, conceptually. What it does that the others cannot, and when it is overkill for a wheel encoder.

THE THEORY THAT MAKES THE TRADE VISIBLE

- Sampling rate, Nyquist, and aliasing. What frequencies are you folding into your signal by sampling too slowly?
- Cutoff frequency and roll-off. What a filter actually removes rather than what you hope it removes.
- **Group delay and phase lag**. The central idea of this problem. How to work out, for each filter above, how far behind reality its output sits.
- Why phase lag inside a feedback loop reduces phase margin, and what that does to a controller tuned without it. This connects directly to problem statement 03.
- Zero-phase filtering, why it is available offline and not in real time, and what that tells you.

RUNNING IT ON A MICROCONTROLLER

- Ring buffers, and fixed window implementations that do not reallocate.

- Integer and fixed-point arithmetic against floating point. What an ESP32 does natively, what an AVR Arduino does not, and what that costs per sample.
- Execution time per sample, and whether your filter still fits inside the loop period at your sample rate.
- Memory: how much RAM a window of N samples across four encoders actually takes.
- Where the filter should live at all — on the microcontroller, or upstream on the Jetson.

3 The Tasks

Get a noisy signal first. Record real encoder velocity or IMU data off a microcontroller if you can, ideally with a motor running nearby so the noise is genuine. If you cannot, synthesise one: a realistic velocity profile, plus quantisation, plus Gaussian noise, plus occasional large spikes.

Same input, every filter
Otherwise the comparison means nothing.

#	TASK
A	Implement four filters on the microcontroller: moving average, EMA, median, and Savitzky-Golay. Run all four on the same input.
B	Measure each one on four axes: noise reduction (standard deviation of the residual), delay in milliseconds , execution time per sample in microseconds, and RAM used. One table, four rows.
C	Show the delay. Feed a step change through each filter and plot input against output on the same time axis. The lag should be visible, not asserted.
D	Put it in a loop. Take the PID from problem statement 03, insert each filter into the feedback path, and show what the added delay does to overshoot and stability. Find the window length at which the loop that was fine becomes unacceptable.
E	Choose. For an encoder velocity feeding a 100 Hz velocity controller, pick one filter and one set of parameters, and defend it against the other three.

4 The Part Most Teams Miss

Before you tune a filter, ask whether filtering is the right answer at all. Three alternatives, each of which is sometimes better:

- **Remove the noise instead of hiding it.** If those spikes are coupled in from a motor cable, a twisted pair costs you nothing in delay and a median filter costs you real delay. Problem statement 01 is the same problem seen from the other end.
- **Sample faster.** Averaging four samples at 1 kHz and averaging four samples at 100 Hz remove similar amounts of noise for very different delays.
- **Send it raw, with a timestamp.** Let the consumer decide how to filter. This costs bandwidth and gives up control, and sometimes it is exactly right. Say when.

Then answer the architectural question directly, and commit to a position: *should filtering live on the microcontroller or on the high-level computer?* There are real arguments both ways — latency, bandwidth, who has the context to know what a good measurement looks like, and what happens to each option when the link between the two hiccups.

5 What to Hand In

ITEM	WHAT IT CONTAINS
Filter code	All four, as they ran on the microcontroller, in the repository.
Comparison table	Task B. Noise reduction, delay in ms, microseconds per sample, RAM. Four rows.
Plots	Step response through each filter, and the closed-loop results from task D.
Your recommendation	Task E plus section 4. Which filter, which parameters, where it runs, and why.
Study notes	Your own notes from section 2. Group delay is the part we will push on.

QUESTIONS YOU SHOULD EXPECT

Your moving average is 20 samples at 200 Hz. How many milliseconds behind reality is its output, and show the reasoning. Why did the median filter beat the moving average on your spiky data? What does Savitzky-Golay preserve that a moving average destroys, and does that matter for a wheel velocity? You added the filter and the controller started oscillating — explain the mechanism, not just the observation. When would you send the raw signal instead of filtering it at all?

Nested Control on a Quadcopter

Four rotors, six degrees of freedom, and four inputs to control all of them. A quadcopter is not one control loop, it is four stacked inside each other, and tuning them in the wrong order is why most people never get one stable.

TYPE	LOOPS	TOOLS	BONUS
Modelling and control	Rate, attitude, velocity, position	MATLAB and Simulink	Rotor failure

1 Why Quadcopters Are a Control Problem

A quadcopter has six degrees of freedom to worry about and four things it can actually command. It cannot translate without tilting first, which means position control is not a loop you can write directly — it is a request passed down to an attitude loop, which passes a request down to a rate loop, which finally asks four motors for four different thrusts.

That stack is the whole problem. Each layer only works if the one beneath it is already fast and well behaved, and the classic failure is somebody tuning the position loop for a snappy response while the rate loop underneath is still sloppy. The aircraft then does something spectacular and nobody can say which loop was at fault.

WHAT YOU SHOULD WALK AWAY WITH

The ability to look at a misbehaving cascaded controller and say which loop is responsible, in what order the loops must be tuned, and how much faster each inner loop has to be than the one above it. This generalises well beyond drones — the same structure runs your arm joints and your drive wheels.

2 What to Study

THE VEHICLE

- Quadcopter configuration, plus and cross, and how rotor direction cancels yaw torque.
- How thrust and torque relate to rotor speed, and the thrust and drag coefficients that connect them.
- **The mixer.** Going from a desired collective thrust plus roll, pitch and yaw torques to four individual motor commands, and back again.
- Rigid-body dynamics: forces, moments, and the inertia matrix.
- Euler angles against quaternions. Gimbal lock, and why flight stacks tend to avoid Euler angles internally.
- Underactuation. Why a quadcopter has to tilt to move, and what that means for how position control has to be built.
- Motor and ESC dynamics: the motor is not instant, and that time constant sits inside your fastest loop.

THE CONTROL STACK

- **Cascaded control.** Position → velocity → attitude → angular rate → mixer → motors. What each layer's input and output actually is.
- **Bandwidth separation.** Why each inner loop must be substantially faster than the loop above it, roughly what factor is usual, and what happens when that separation is not respected.
- Loop rates. Why the rate loop runs at hundreds of hertz and the position loop does not need to.
- Tuning order, and why it can only be innermost first.
- Gravity compensation and feedforward in the thrust channel.
- Attitude estimation: gyroscope integration drifts, accelerometers are noisy and useless while accelerating, and how a complementary or Kalman filter combines them. This connects to problem statement 06.
- Actuator saturation and control allocation. What the mixer does when it is asked for more thrust than exists, and why prioritising attitude over altitude is usually the right call.

3 The Model

Everyone works from the same aircraft so results can be compared. Build it, or use an existing quadcopter model and state which.

```
mass 1.2 kg · arm length 0.23 m · cross configuration
Ixx = Iyy = 0.011 kg·m2 · Izz = 0.021 kg·m2
max thrust per rotor 8 N · motor time constant 30 ms
```

```
suggested loop rates: rate 500 Hz · attitude 250 Hz · velocity 100 Hz · position 50 Hz
```

Those loop rates are a starting point, not a specification. Part of the work is showing whether they are justified.

TEST IT IN SIMULATION

All of this is meant to be built and run in **MATLAB and Simulink**. Nothing here needs an aircraft, and nothing should be claimed without a plot behind it. If you would rather work in Python, that is fine, but say so and make your model reproducible either way.

4 The Tasks

TASK

- A Build the mixer.** Given desired thrust and three torques, produce four motor commands. Show it works in both directions, and show what it does when a command exceeds what the rotors can deliver.
- B Tune the rate loop** for roll, pitch and yaw. Report gains and step responses. This is the loop everything else stands on, so spend real time here.
- C Close the attitude loop** around it, then the velocity loop, then position. One at a time, gains and plots at every stage. State the bandwidth of each loop and the ratio between neighbours.
- D Break the separation deliberately.** Make the attitude loop as fast as the rate loop, or faster. Show what happens and explain the mechanism. This is the single most useful plot you will produce.
- E Add reality.** Motor saturation, the 30 ms motor lag, gyro noise, and 10 ms of sensor delay. Retune as needed and report what changed and what you had to give up.
- F Fly a trajectory.** Take off, move 5 m in x, hold, then land. Plot position tracking and motor commands throughout. Motor commands hitting their limits during a routine manoeuvre is a finding worth reporting.

5 Extension: Losing a Rotor **BONUS**

Prior art

This has been a full Inter IIT problem statement in its own right. Treat it as a direction to explore, not something to finish.

Kill one rotor mid-flight and watch what your controller does. It will not go well, and understanding *why* is the point: with three rotors you no longer have four independent control authorities, and something has to be sacrificed.

Then think about what a controller could do instead of failing. The known result in this area is counter-intuitive and worth arriving at yourself: something has to be given up, and the aircraft can remain controllable in the axes that matter if you are willing to abandon one of them entirely. Work out which one, and why that is the affordable one to lose.

We are not expecting a working fault-tolerant controller. We are expecting you to demonstrate the failure, reason clearly about what control authority remains, and propose an approach you can defend.

6 What to Hand In

ITEM

WHAT IT CONTAINS

Model and code	Everything needed to reproduce every plot, in the repository.
Gains table	Every loop, its gains, its rate, its measured bandwidth, and the ratio to the loop above it.
Plots	All six tasks, each with a sentence on what it shows. Task D in particular.
Tuning account	

	How you actually tuned each loop, in order, including what you tried that did not work.
Failure note	Section 5, if attempted. What broke, what authority was left, and what you would do about it.

QUESTIONS YOU SHOULD EXPECT

Why did you tune the rate loop first, and what would have gone wrong the other way round? What is the bandwidth of your attitude loop, and how did you measure it rather than guess it? Your quadcopter drifts slowly in position while holding attitude perfectly — which loop is at fault? What does your mixer do when it is asked for more thrust than the rotors have, and why did you choose that behaviour? With one rotor gone, what exactly can you no longer control?